

CCSAR

Instructions: Language of the Computer Part I

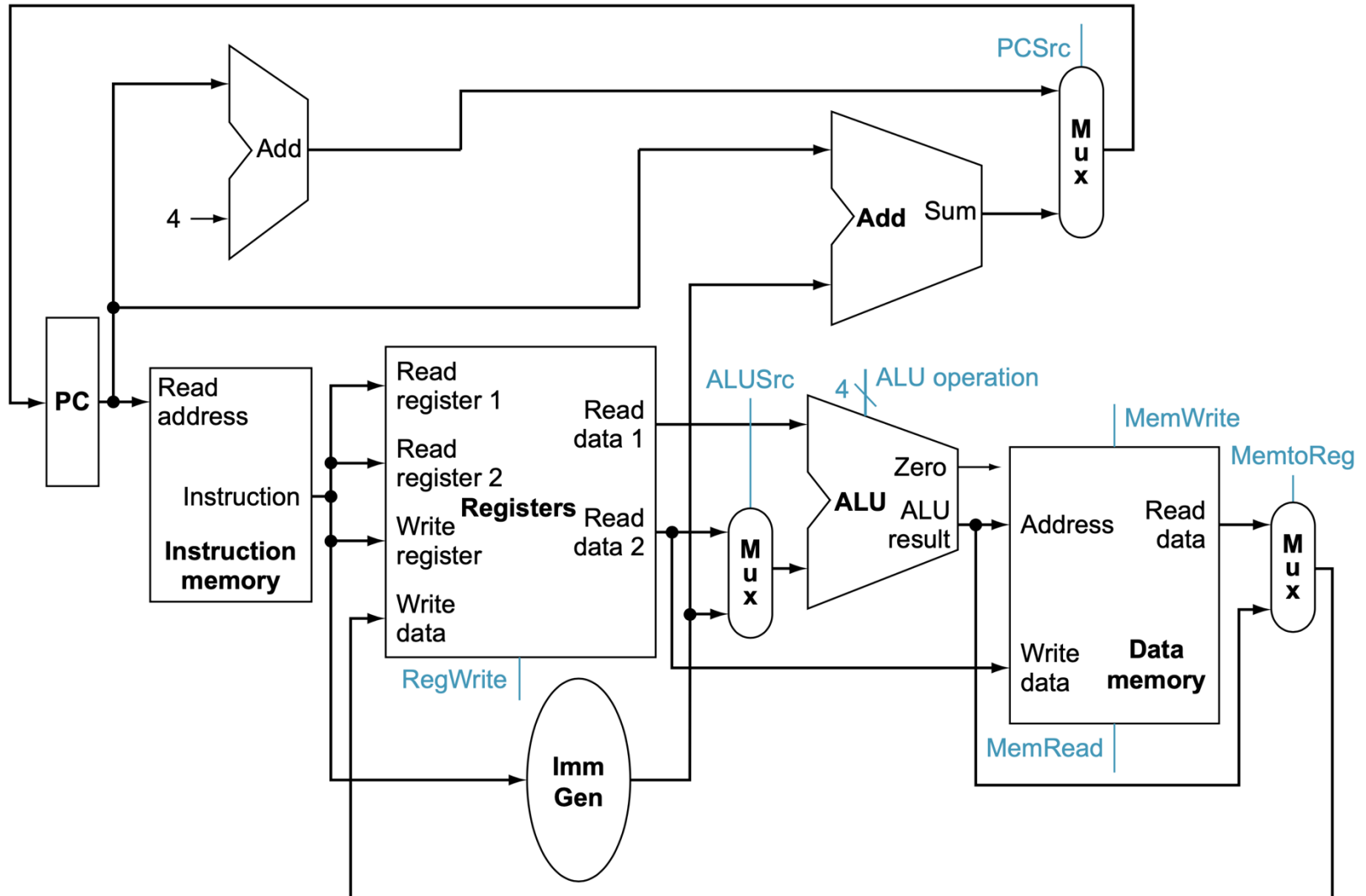
Arquiteturas de Computadores para Sistemas Críticos
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Goal for the Next Lectures

- **In this and the next lectures, we will cross the boundary between software and hardware**
 - At the software side, we will look at C as a high-level programming language
 - At the hardware side, we will use the RISC-V ISA to demonstrate assembly and machine language
- **However, these concepts are applicable to any programming language, and any instruction set architecture**

Big Picture: Building a Processor



Definitions

■ Instruction Set Architecture (ISA)

- The **interface** between software and the processor
- Defines the instructions, registers, data types, memory access and control flow visible to software
- E.g., RISC-V, ARM, x86-64, MIPS

■ Microarchitecture

- **Implementation** of the architecture
- E.g., pipelining, out-of-order execution, cache sizes, core frequency, ...

Definitions

- **Assembly code**
 - A **text representation** of machine instructions
- **Machine code**
 - The **binary encoding** of instructions executed by a processor

The C Programming Language

- **Why C?**

- Close to the hardware
- Explicit memory management
- Efficient and predictable
- Widely used in operating systems and embedded software

- **But processors execute machine instructions, not C**

Levels of Program Code

■ High-level language

- Level of abstraction closer to problem domain
- Provides for productivity and portability

High-level
language
program
(in C)

```
swap(size_t v[], size_t k)
{
    size_t temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compiler

■ Assembly language

- Textual representation of instructions

Assembly
language
program
(for RISC-V)

```
swap:
    slli x6, x11, 3
    add  x6, x10, x6
    lw   x5, 0(x6)
    lw   x7, 4(x6)
    sw   x7, 0(x6)
    sw   x5, 4(x6)
    jalr x0, 0(x1)
```

Assembler

■ Hardware representation

- Binary digits (bits)
- Encoded instructions and data

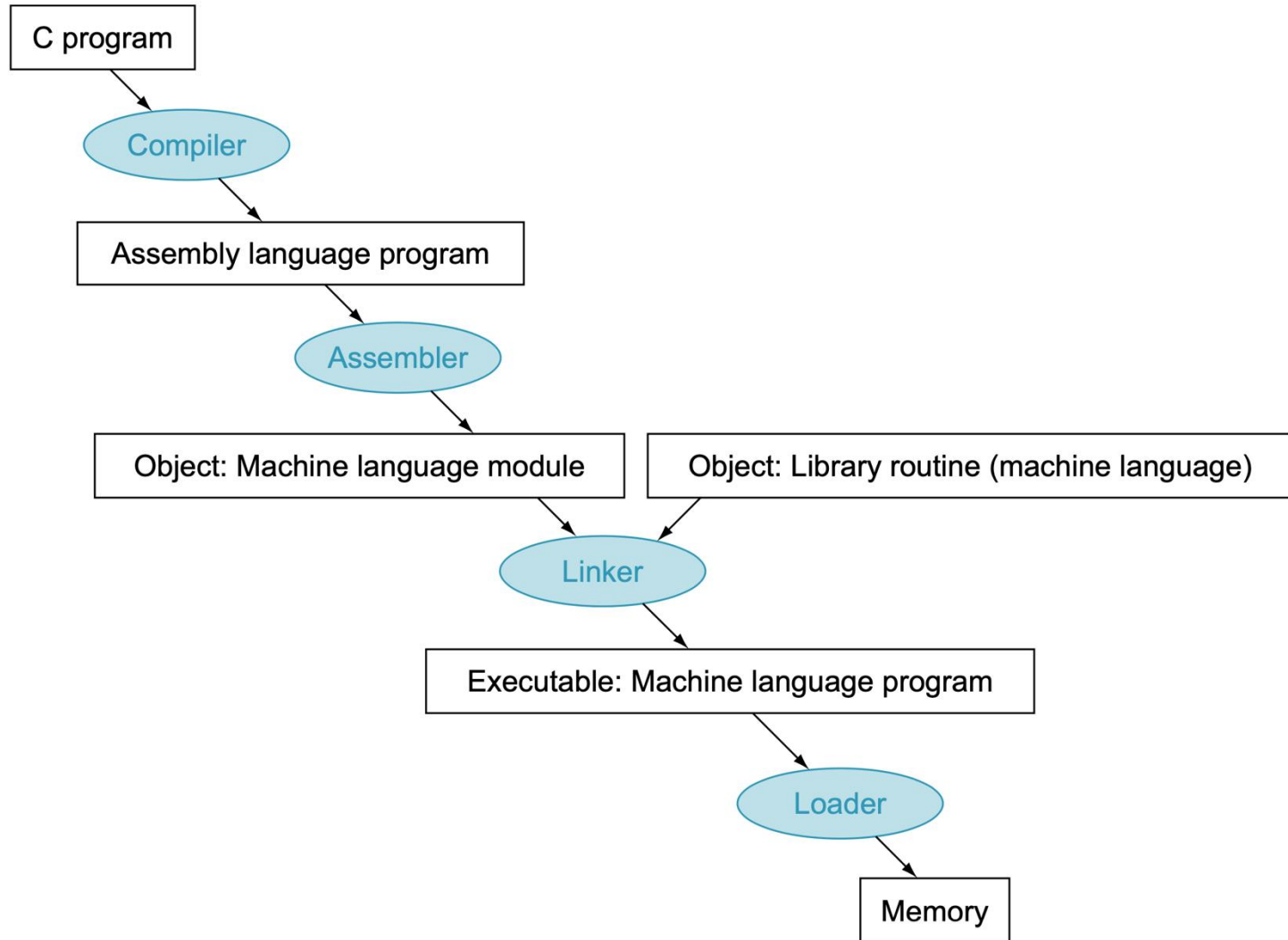
Binary machine
language
program
(for RISC-V)

```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000001000000001100111
```

Levels of Program Code

- **Recompiling everything would be wasteful**
 - Standard library routines rarely change
 - Programmers would repeatedly compile and assemble the same code
- **Instead, programs are built from independently compiled modules**
 - The **linker** combines object files and resolves their references
 - It produces a single **executable file**
- **Before execution, the executable must be **loaded into memory****

Translating and Starting a C Program



Instruction Set Architecture

- **To control a computer's hardware, software must speak its language**
 - Instructions are the basic operations the processor can perform
- **The instruction set architecture (ISA) defines this language:**
 - Instructions, Registers, Data types, Memory access, Control flow
- **Different ISAs are more like dialects than different languages**
 - They share many fundamental concepts and operations, but differ in syntax and details
 - Learning one ISA makes it easier to learn another

CISC vs RISC

- **CISC and RISC represent different approaches to designing an ISA**
 - RISC emphasizes simplicity and regularity, making instructions easier to decode and execute efficiently

CISC	RISC
More complex instructions	Simpler instructions
Many addressing modes	Fewer addressing modes
Variable-length instructions are common	Fixed-length instructions are common
More work per instruction	Simpler operations per instruction
Hardware may be more complex	Hardware can be simpler and more regular

Complex Instruction Set (CISC)

- **Perform more work with fewer instructions**
 - A single instruction may combine several low-level operations
- **Complexity is shifted to the hardware**
 - Variable-length instructions
 - Multiple addressing modes
 - More complex instruction decoding
 - Instructions may require multiple clock cycles
- **Examples: VAX, Motorola 68000, Intel x86**
 - Some CISC ISAs even included highly specialized instructions — the VAX, for example, had instructions for polynomial evaluation

Reduced Instruction Set (RISC)

- **A small set of simple, general-purpose instructions**
 - Designed for efficient implementation, fast instruction execution, and effective pipelining
- **Complexity is shifted to software**
 - More instructions may be needed to perform the same task
 - Compilers do more of the work to combine simple instructions into efficient programs
 - Simple, regular instructions make hardware easier to implement and pipeline
- **Examples: RISC-V, ARM, MIPS, IBM PowerPC**

The RISC-V Instruction Set

- **Used as the example throughout the course**
 - Simple enough to be used in computer architecture courses
- **Developed at UC Berkeley as an open ISA specification**
 - Now managed by the RISC-V International (<https://riscv.org/>)
 - Open standard, modular design, fixed-length instruction formats, load/store architecture
- **Wide industry adoption**
 - Growing ecosystem of hardware and software implementations
 - Used in diverse applications from IoT devices to data centers

RISC-V Core Principles

- **RISC (Reduced Instruction Set Computing)**
 - RV32I emphasizes a smaller, optimized set of common instructions rather than a large, complex instruction set
- **Load-Store architecture**
 - Load and store instructions access memory, while other instructions work with CPU registers
- **Fixed-length instructions**
 - In RV32I, all instructions are 32-bit long, simplifying instruction fetching and decoding in hardware

RISC-V Core Principles

- **Simple does not mean limited**
 - RV32I provides the basic operations needed to build arbitrary programs
- **Modular extensions**
 - The instruction set is designed to be extensible, allowing optional extensions to be added for specific needs without making the base set complex
 - E.g., floating-point (F) or atomic (A) operations

RISC-V Instruction Set

- **RISC-V handles processors of different sizes with separate but consistent instruction sets**
 - **RV32** – 32-bit RISC-V with 32 general-purpose registers
 - **RV64** – 64-bit RISC-V with 32 general-purpose registers
 - **RV32E** – Reduced 32-bit RISC-V with 16 general-purpose registers
- **We'll be focusing on RV32, but the other instruction sets work in a similar way**
- **The base integer instruction set for RV32 is RV32I (“I” stands for integer)**
 - RV32I contains the essential RISC-V instructions, including arithmetic, memory access, and flow control

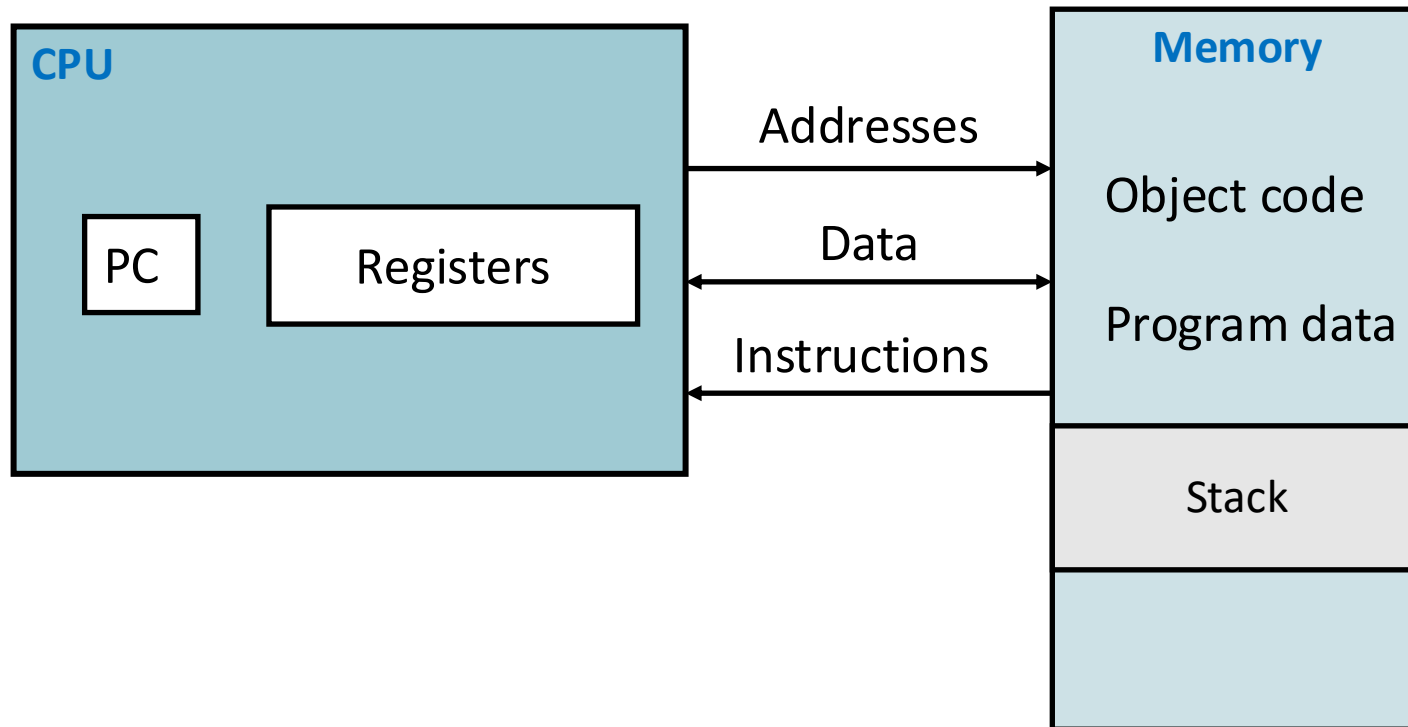
RISC-V Simplified Implementation

- **We will consider a simplified implementation of the RISC-V base instruction set architecture to illustrate:**
 1. How it determines many aspects of the implementation
 2. How the choice of different implementation strategies affects the clock rate and CPI for the computer
- **Using a subset of the core RV32I instruction set**
 - Arithmetic-logical instructions: `add`, `sub`, `and`, `or`, `slt`, ...
 - Memory reference instructions: `lw`(load word), `sw`(store word)
 - Control flow instructions: `beq`(branch equal), `bne` (branch not equal), `jal` (jump and link), ...

RISC-V Simplified Implementation

- **See the RISC-V Reference Data tear-out card in Moodle**
- **Try out the several examples in a basic interpreter**
 - Available at <http://www.dei.isep.ipp.pt/~luis/ccsar/interpreter/>
- **If you intend to build a cross-toolchain for compiling RV32 code on Linux, this page explains how**
 - See <https://www.dei.isep.ipp.pt/~luis/ccsar/toolchain/>

Assembly Programmer's View



Programmer-visible state

- **PC: Program Counter**
 - Address of the current instruction
- **Registers**
 - Heavily used program data

Memory

- Byte addressable array
- Code and user data
- Stack to support procedures

Arithmetic Operations

- **Example: adding two values**
 - Two sources and one destination, all in registers

add x5, x6, x7 **# a = b + c**

- **All arithmetic operations have this form**

Design Principle 1: Simplicity favors regularity

Regularity makes implementation simpler

Simplicity enables higher performance at lower cost

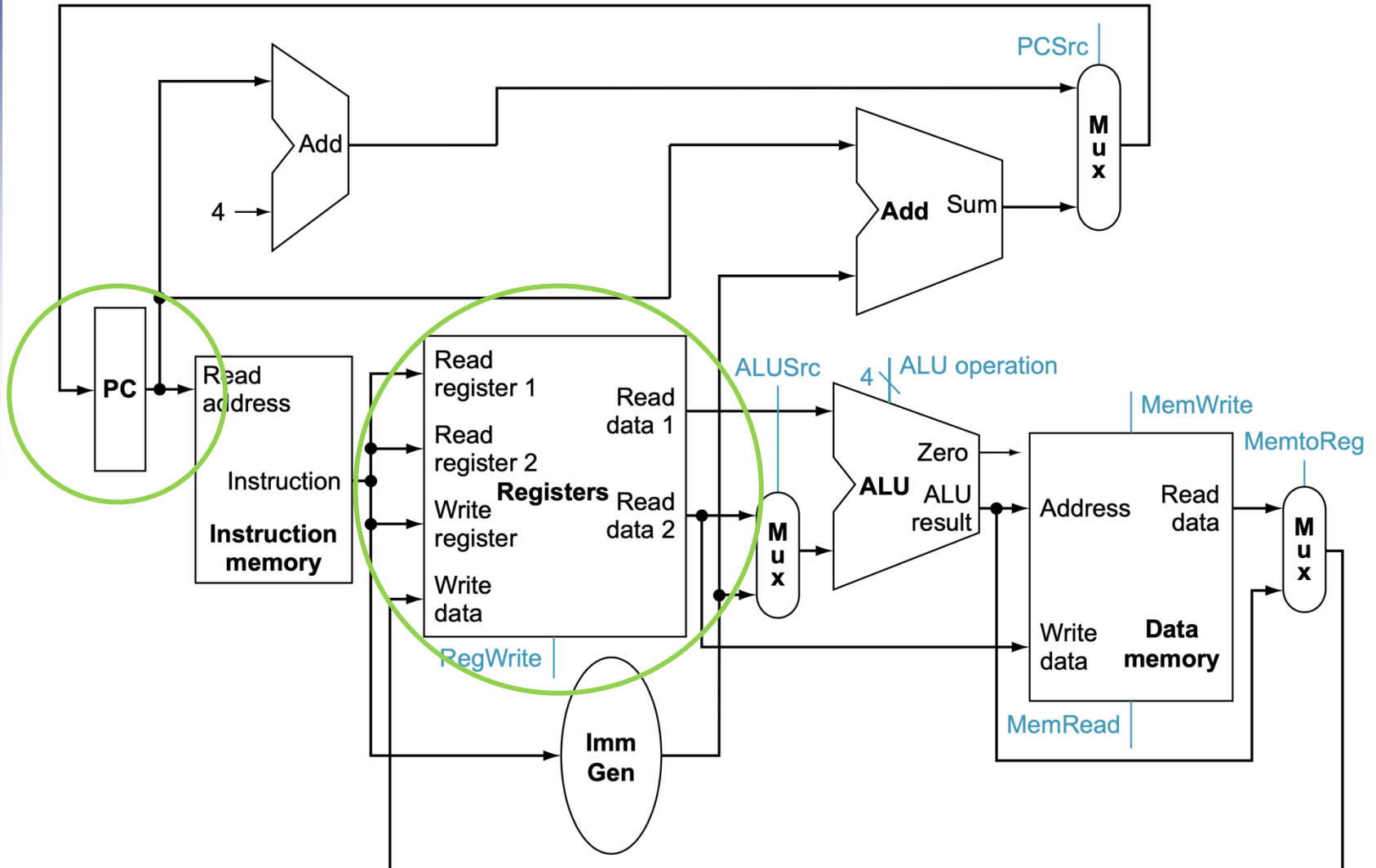
Register Operands

- Arithmetic instructions use **register operands**
- **RV32 has 32 general-purpose registers: x0 to x31**
 - These registers are 32-bit wide
 - They form an integral part of the CPU design and accessing them is easy and fast
 - That means much faster than regular memory

Design Principle 2: Smaller is faster

The designer must balance the craving of programs for more registers with the designer's desire to keep the clock cycle fast

RISC-V Register File



Register Operands

- **x0 is hard-wired to 0 (zero)**
 - Values written to it **are discarded**
 - Its main role is to simplify the instruction set by offering useful variations
 - E.g., move between registers: `add x2, x1, x0`
- **You can use the other registers as needed, but software follows conventional roles**
 - These conventions allow independently written code to work together
- **The ABI (Application Binary Interface) and calling conventions will be covered in the next lecture**
 - The ISA defines the registers; the ABI defines conventions for using them

Register Operand Example

- **C code:**

```
f = (g + h) - (i + j);
```

- **Compiled RISC-V code:**

- f, ..., j in x19, ..., x23

```
add x5, x20, x21  
add x6, x22, x23  
sub x19, x5, x6
```

Immediate Operands

- **What if one operand is a constant?**
- **Loading the constant into a register would require an extra instruction**
- **Immediate operands provide a simpler alternative**
 - Small constants can be **encoded directly in the instruction**
- **So, this is not just a convenience**
 - It is an example of a **design principle**

Immediate Operands

- **Constant data specified in an instruction**
 - The immediate value is **sign-extended from 12 to 32 bits**

`addi x22, x22, 4` **# x22 = x22 + 4**

- **No subtract immediate instruction**
 - Just use a negative constant

`addi x22, x22, -1`

Design Principle 3: Make the common case fast

Small constants are common

Immediate operands avoid an extra instruction to obtain a small constant

This can improve both performance and energy efficiency

Arithmetic Instructions

add	x1, x2, x3	$x1 = x2 + x3$	add
addi	x1, x2, imm	$x1 = x2 + \text{imm}$	add immediate
sub	x1, x2, x3	$x1 = x2 - x3$	subtract

■ Reference — not memorization

- These tables are provided for consultation, not memorization
- Focus on understanding instruction semantics, operands, and usage
- You may refer to these tables when needed

Bitwise Operations

- **Instructions for bitwise manipulation**
 - Shift operations can also be used for efficient multiplication or division by powers of two

Operation	C	RISC-V
Shift left logical	<<	sll, slli
Shift right logical	>>	srl, srli
Shift right arithmetic	>>	sra, srai
Bitwise AND	&	and, andi
Bitwise OR		or, ori
Bitwise XOR	^	xor, xori
Bitwise NOT	~	pseudo-instruction not

Shift Operations

■ Shift left logical

- Shift left and **fill with 0 bits**
- `sll` by i bits effectively multiplies by 2^i

`slli x3, x4, 4` **# x3 = x4 << 4**

■ Shift right logical / arithmetic

- Shift right and **fill with 0 bits** (logical)
- Shift right and **replicate MSB** (arithmetic)
- `srl` by i bits divides by 2^i (unsigned only)

`srl x5, x6, x7` **# x5 = x6 >> x7**

Bitwise Operations

- **RISC-V keeps the base ISA minimal**
 - Bitwise NOT is expressible with an existing instruction (`xori rd, rs, -1`)
- **So, there is no separate machine instruction for it in RV32I**
- **Assemblers provide `not` as a pseudo-instruction**
 - (more on this in the next lecture)

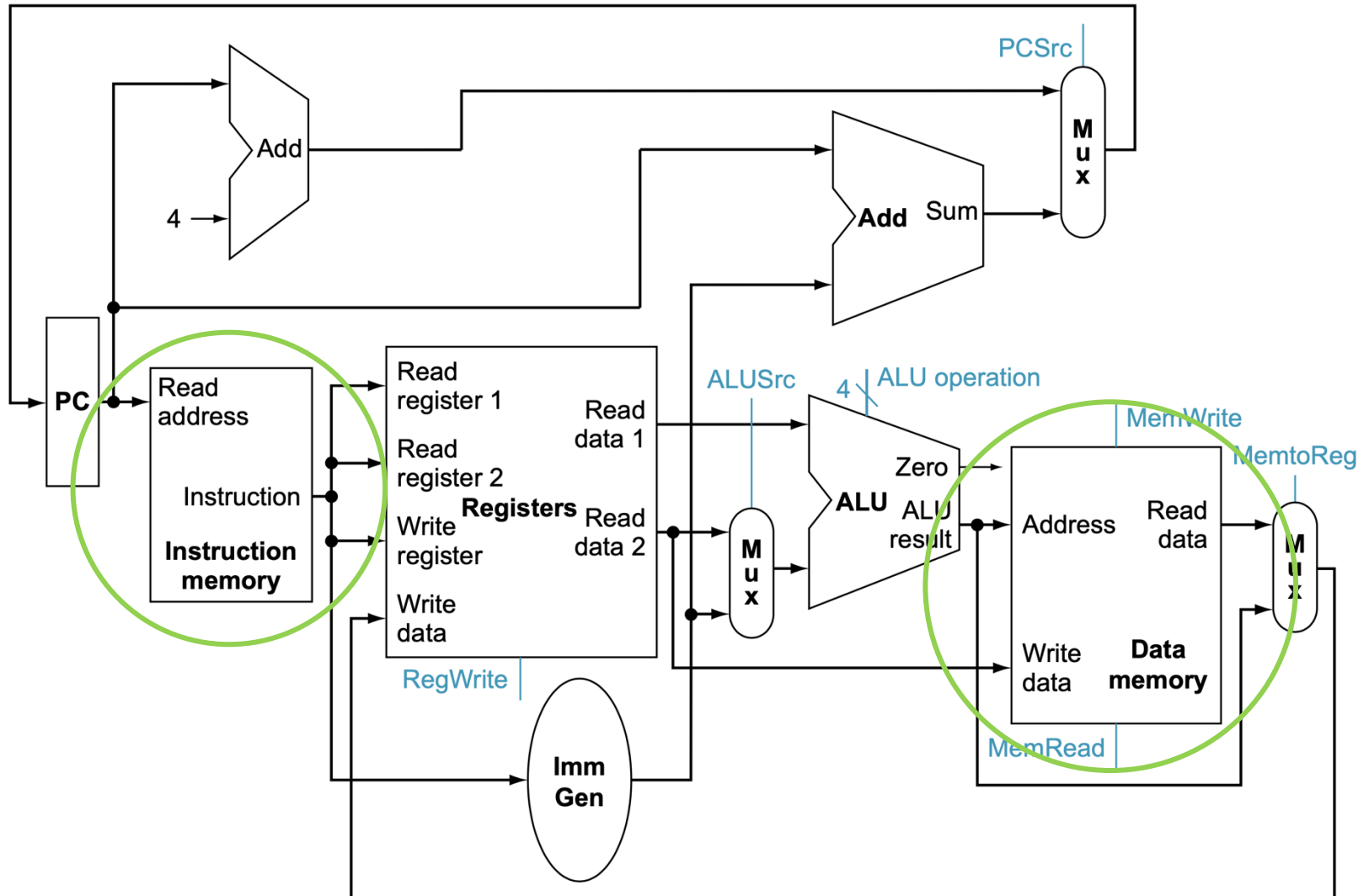
Bitwise Instructions

and	x1, x2, x3	x1= x2 & x3	bitwise and
andi	x1, x2, imm	x1= x2 & imm	bitwise and immediate
or	x1, x2, x3	x1= x2 x3	bitwise or
ori	x1, x2, imm	x1= x2 imm	bitwise or immediate
xor	x1, x2, x3	x1= x2 ^ x3	bitwise xor
xori	x1, x2, imm	x1= x2 ^ imm	bitwise xor immediate
sll	x1, x2, x3	x1= x2 << x3	shift left logical
slli	x1, x2, imm	x1= x2 << imm	shift left logical immediate
srl	x1, x2, x3	x1= x2 >> x3	shift right logical
srlr	x1, x2, imm	x1= x2 >> imm	shift right logical immediate
sra	x1, x2, x3	x1= x2 >> x3	shift right arithmetic
srai	x1, x2, imm	x1= x2 >> imm	shift right arithmetic immediate

Registers vs. Memory

- **Registers are faster to access than memory**
 - This speed advantage is crucial for maximizing the performance of CPUs and executing instructions efficiently
- **Compilers try to keep frequently used values in registers**
 - Values that cannot remain in registers may be spilled to memory
 - Register optimization is important!
- **Operating on memory data requires loads and stores**
 - More instructions to be executed
 - It is also a lot slower (memory access latency, bandwidth limitations, concurrency, ...)

RISC-V Memory



Memory Operands

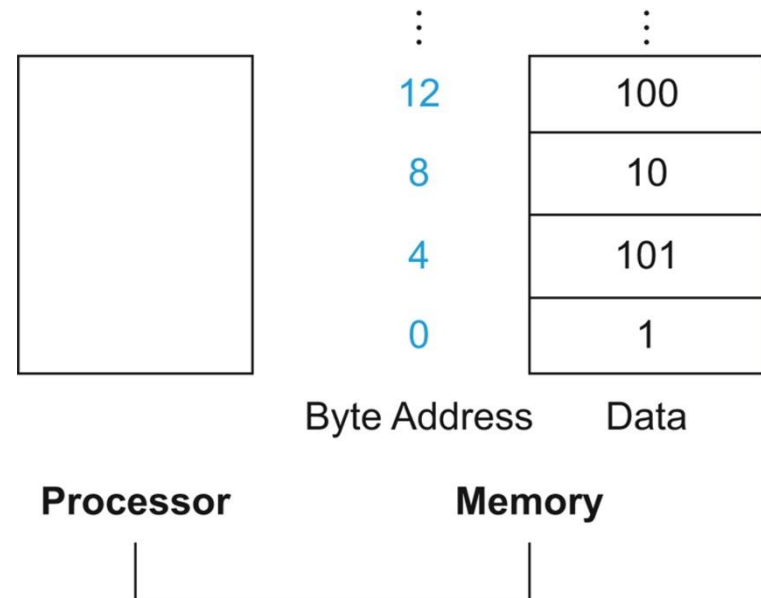
- **Main memory for composite data**
 - Used to store arrays, structures, and dynamically allocated data
- **Byte-addressable memory**
 - Memory can be viewed as a large, one-dimensional array of bytes, with the address identifying the byte at each position
- **Word alignment**
 - In RV32 systems, 4-byte words are typically **aligned to 4-byte boundaries**
 - Misaligned accesses may require additional handling and may be less efficient or unsupported, depending on the implementation

Memory Operands

- **RISC-V systems are typically little-endian**
 - The least-significant byte (LSB) is stored at the lowest memory address
- **RISC-V also supports big-endian systems**
 - The byte ordering is an implementation choice
 - Therefore, the endianness of a RISC-V system is determined by the implementation

Accessing memory

- **Data transfer instructions**
 - Transfer data between memory and registers
- **RISC-V commonly uses displacement addressing**
 - Data is at the memory location whose address is the **sum of a register and a constant** in the instruction



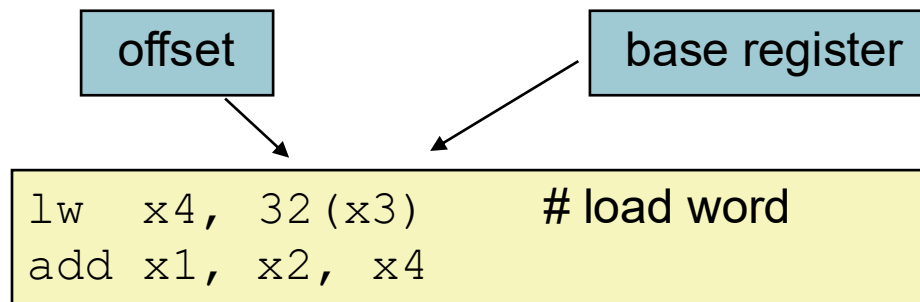
Memory Operand: Example 1

- **C code:**

```
g = h + A[8];
```

- **Compiled RISC-V code:**

- g in x1, h in x2, base address of A in x3
- Index 8 requires offset of 32
 - 4 bytes per word



Memory Operand: Example 2

- **C code:**

```
A[12] = h + A[8];
```

- **Compiled RISC-V code:**

- h in x2, base address of A in x3
- Index 8 requires offset of 32

```
lw  x4, 32(x3)    # load word
add x4, x2, x4
sw  x4, 48(x3)    # store word
```

Byte/Halfword Operations

- The RISC-V ISA has explicit instructions to load and store 16-bit (*halfwords*) and 8-bit quantities (*bytes*)

`lb rd, offset(rs1)`

- Sign extend to 32 bits in `rd`

`lh rd, offset(rs1)`

`lbu rd, offset(rs1)`

- Zero extend to 32 bits in `rd`

`lhu rd, offset(rs1)`

`sb rs2, offset(rs1)`

- Store just rightmost byte/halfword

`sh rs2, offset(rs1)`

Why not unsigned store?

- **When storing data, the concept of “unsigned” is irrelevant**
 - Store instructions write the raw binary representation from a register to memory, regardless of whether the value is interpreted as signed or unsigned
- **Sign matters when loading data**
 - The **interpretation** of the stored bits as signed or unsigned matters when the data is loaded into a register, because different load instructions may sign-extend or zero-extend the value

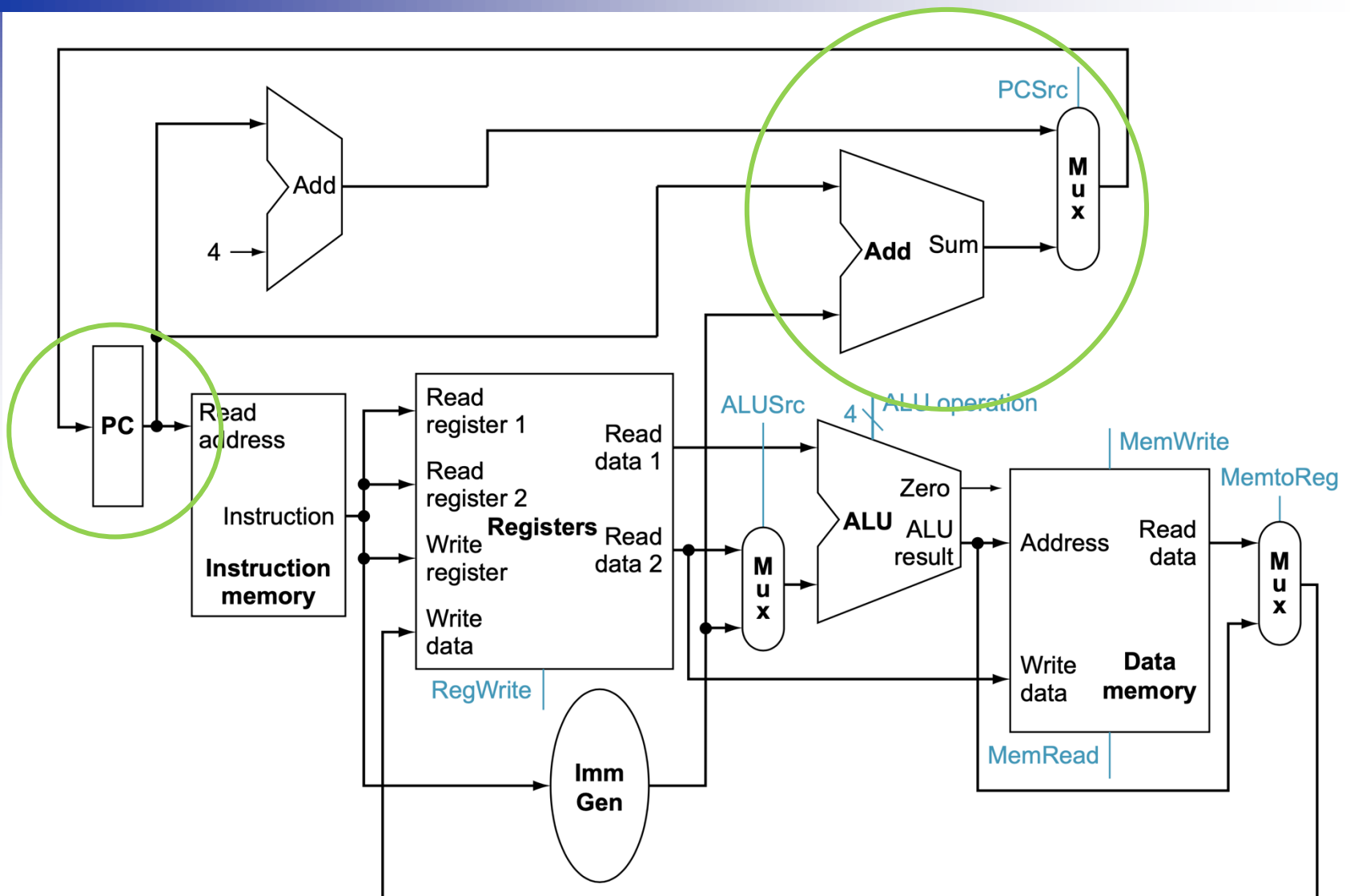
Data Transfer Instructions

lw x1, imm(x2)	$x1 = \text{MEM}[x2 + \text{imm}]$	load word from memory
lh x1, imm(x2)	$x1 = \text{MEM}[x2 + \text{imm}]$	load half word (2 bytes)
lhu x1, imm(x2)	$x1 = \text{MEM}[x2 + \text{imm}]$	load half word unsigned
lb x1, imm(x2)	$x1 = \text{MEM}[x2 + \text{imm}]$	load byte
lbu x1, imm(x2)	$x1 = \text{MEM}[x2 + \text{imm}]$	load byte unsigned
sw x1, imm(x2)	$\text{MEM}[x2 + \text{imm}] = x1$	store word to memory
sh x1, imm(x2)	$\text{MEM}[x2 + \text{imm}] = x1$	store half word (2 bytes)
sb x1, imm(x2)	$\text{MEM}[x2 + \text{imm}] = x1$	store byte

Conditional Operations

- **Branch to a **labeled instruction** when a condition is true**
 - Otherwise, execution continues sequentially with the next instruction
- **The assembler handles branch address calculations**
 - It saves both the compiler and assembly programmer from manual address calculations, just as it does for data addresses in load and store instructions
- **Branch targets use signed PC-relative offsets**
 - The branch offset is a signed 13-bit value, with the least-significant bit implicitly zero
 - This gives a byte-offset range of -4096 to $+4094$ bytes

Big Picture: Building a Processor



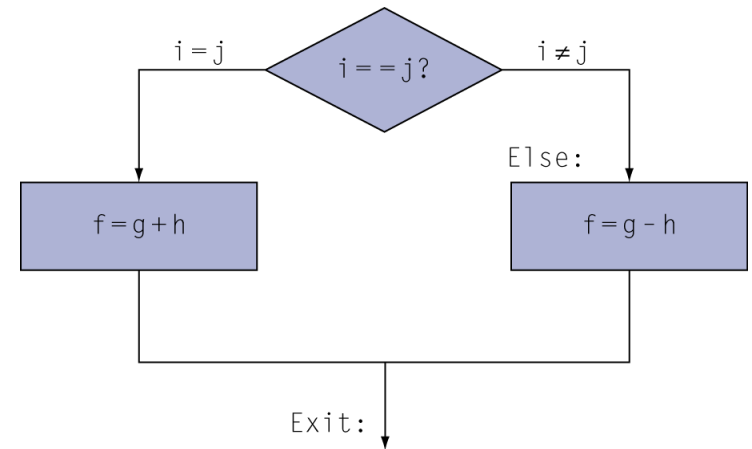
Conditional Operations

- `beq rs1, rs2, L1`
 - If (`rs1 == rs2`) branch to instruction labeled L1
- `bne rs1, rs2, L1`
 - If (`rs1 != rs2`) branch to instruction labeled L1
- `blt rs1, rs2, L1`
 - if (`rs1 < rs2`) branch to instruction labeled L1
- `bge rs1, rs2, L1`
 - if (`rs1 >= rs2`) branch to instruction labeled L1

Compiling If Statements

■ C code:

```
if (i==j)
    f = g+h;
else
    f = g-h;
```



■ Compiled RISC-V code:

- `f, g, ...` in `x19, x20, ...`

```
        bne x22, x33, Else
        add x19, x20, x21
        beq x0, x0, Exit      # unconditional
Else:    sub x19, x20, x21
Exit:    ...
```

Assembler calculates addresses

Compiling Loop Statements

- **C code:**

```
while (save[i] == k)
    i += 1;
```

- **Compiled RISC-V code:**

- i in $x22$, k in $x24$, address of $save$ in $x25$

```
Loop: slli x10, x22, 2
      add  x10, x10, x25
      lw   x9, 0(x10)
      bne  x9, x24, Exit
      addi x22, x22, 1
      beq  x0, x0, Loop
Exit: ...
```

Signed vs Unsigned Branching

- **Signed comparison:** `blt`, `bge`
- **Unsigned comparison:** `bltu`, `bgeu`
- **Example**
 - `x22 = 1111 1111 1111 1111 1111 1111 1111 1111`
 - `x23 = 0000 0000 0000 0000 0000 0000 0000 0001`
 - `x22 < x23` // signed
 - `-1 < +1`
 - `x22 > x23` // unsigned
 - `+4,294,967,295 > +1`

More Conditional Operations

- **Set result to 1 if a condition is true**
 - Otherwise, set to 0
- `slt rd, rs1, rs2`
 - `if (rs1 < rs2) rd = 1; else rd = 0;`
- `slti rd, rs1, imm`
 - `if (rs1 < imm) rd = 1; else rd = 0;`
- **Usually used in combination with beq, bne**

```
slt x3, x1, x2    # if (x1 < x2)
bne x3, x0, L      # branch to L
```

Conditional Instructions

beq	x1, x2, L	if(x1 == x2) goto L	branch if equal
bne	x1, x2, L	if(x1 != x2) goto L	branch if not equal
blt	x1, x2, L	if(x1 < x2) goto L	branch if less than
bge	x1, x2, L	if(x1 >= x2) goto L	branch if greater than
bltu	x1, x2, L	if(x1 < x2) goto L	branch if less than unsigned
bgeu	x1, x2, L	if(x1 >= x2) goto L	branch if greater than or equal unsigned
slt	x3, x1, x2	if (x1 < x2) x3 = 1; else x3 = 0	Set if less than
sltu	x3, x1, x2	if (x1 < x2) x3 = 1; else x3 = 0	Set if less than unsigned
slti	x3, x1, imm	if (x1 < imm) x3 = 1; else x3 = 0	Set if less than immediate
sltiu	x3, x1, imm	if (x1 < imm) x3 = 1; else x3 = 0	Set if less than immediate unsigned

Concluding Remarks

- **The ISA is the interface between software and hardware**
 - Programs written in high-level languages are translated into machine instructions
 - RISC-V provides a simple and regular instruction set
 - Complex operations are built from combinations of simple instructions
 - Registers provide fast access to operands; memory is accessed explicitly through load/store instructions
 - Branches and jumps implement control flow
- **The key idea**
 - Understanding the ISA is the first step towards understanding how a processor executes a program